

Formalizing the Sauer–Shelah Lemma in Lean 4

Author: Zeyu Bian · DSC 190/291, Learning Theory · Spring 2026 **Artifact:** Project/Project/SauerShelah.lean (237 lines, builds against mathlib, no `sorry`).

This report has three parts, in the order we approach the theorem formalization:

1. **The proof on paper** — a precise, self-contained natural-language proof.
2. **The formalization** — how each step of the paper proof becomes Lean code *from scratch* (we do not invoke mathlib’s Sauer–Shelah results).
3. **Verification** — how a human (or machine) confirms the Lean artifact really proves the claimed theorem and nothing weaker. —

0. Statement

Let X be a finite ground set with $|X| = n$. A **concept class** is a family $\mathcal{A} \subseteq 2^X$ of subsets of X (each concept identified with the set of points it labels 1).

For $S \subseteq X$, we say $\mathcal{A}$ **shatters** S if every subset $U \subseteq S$ is realized as a trace:

$$\mathcal{A} \text{ shatters } S \iff \forall U \subseteq S, \exists A \in \mathcal{A}, A \cap S = U.$$

Equivalently, the $2^{|S|}$ possible labelings of S all occur among members of $\mathcal{A}$.

The **VC dimension** of $\mathcal{A}$ is the size of the largest shattered set, $d = \text{VCdim}(\mathcal{A}) = \max\{|S| : \mathcal{A} \text{ shatters } S\}$.

We aim to formalize the following lemma in Lean4.

$$\textbf{Sauer–Shelah Lemma. } |\mathcal{A}| \leq \sum_{k=0}^d \binom{n}{k}.$$

1. The proof on paper

The proof factors into two independent inequalities:

$$|\mathcal{A}| \stackrel{\text{(I)}}{\leq} \#\{\text{sets shattered by } \mathcal{A}\} \stackrel{\text{(II)}}{\leq} \sum_{k=0}^d \binom{n}{k}.$$

Write $\text{Tr}(\mathcal{A}) = \{S \subseteq X : \mathcal{A} \text{ shatters } S\}$ for the set of shattered sets (the “trace family”).

(I) Pajor’s lemma: $|\mathcal{A}| \leq |\text{Tr}(\mathcal{A})|$

A family shatters at least as many sets as it has members. Proof by induction on the ground set X .

Base case $X = \emptyset$. Every member of $\mathcal{A}$ is $\subseteq \emptyset$, so $\mathcal{A} \subseteq \{\emptyset\}$, giving $|\mathcal{A}| \in \{0, 1\}$. If $\mathcal{A} = \emptyset$ then $0 \leq |\text{Tr}|$ trivially. If $\mathcal{A} = \{\emptyset\}$ then $\mathcal{A}$ shatters $\emptyset$ (the only subset of $\emptyset$ is $\emptyset = \emptyset \cap \emptyset$), so $|\text{Tr}| \geq 1$. Either way $|\mathcal{A}| \leq |\text{Tr}(\mathcal{A})|$.

Inductive step $X = X' \sqcup \{a\}$ (with $a \notin X'$). Split and project on a . Define three families over the smaller ground set X' :

- $\mathcal{A}_a = \{A \in \mathcal{A} : a \in A\}$, the members containing a ;

- $M = \{A \setminus \{a\} : A \in \mathcal{A}_a\}$, those members with a deleted;
- $\mathcal{A}_{\bar{a}} = \{A \in \mathcal{A} : a \notin A\}$, the members avoiding a .

Set $P = M \cup \mathcal{A}_{\bar{a}}$ and $Q = M \cap \mathcal{A}_{\bar{a}}$. All of P, Q are families of subsets of X' .

Counting identity $|\mathcal{A}| = |P| + |Q|$. Deleting a is injective on $\mathcal{A}_a$ (each such A equals $\{a\} \cup (A \setminus \{a\})$), so $|M| = |\mathcal{A}_a|$. Splitting $\mathcal{A}$ by whether a is present gives $|\mathcal{A}| = |\mathcal{A}_a| + |\mathcal{A}_{\bar{a}}| = |M| + |\mathcal{A}_{\bar{a}}|$. Inclusion–exclusion on $M, \mathcal{A}_{\bar{a}}$ gives $|P| + |Q| = |M \cup \mathcal{A}_{\bar{a}}| + |M \cap \mathcal{A}_{\bar{a}}| = |M| + |\mathcal{A}_{\bar{a}}|$. Hence $|\mathcal{A}| = |P| + |Q|$. (*Combinatorially: a set $B \subseteq X'$ is counted once in P if exactly one of $B, \{a\} \cup B$ lies in $\mathcal{A}$, and once more in Q if both do — exactly matching how the two preimages B and $\{a\} \cup B$ contribute to $|\mathcal{A}|$.)*

Transferring shattered sets. Two claims relate traces over X' to traces over X :

- **(A)** If P shatters $S \subseteq X'$, then $\mathcal{A}$ shatters S . *Reason:* given $U \subseteq S$, P supplies B with $B \cap S = U$. If $B \in \mathcal{A}_{\bar{a}} \subseteq \mathcal{A}$, take $A = B$. If $B \in M$, then $\{a\} \cup B \in \mathcal{A}$ and, since $a \notin S$, $(\{a\} \cup B) \cap S = B \cap S = U$.
- **(B)** If Q shatters $S \subseteq X'$, then $\mathcal{A}$ shatters $\{a\} \cup S$. *Reason:* take $U \subseteq \{a\} \cup S$. If $a \notin U$: $U \subseteq S$; $Q \subseteq \mathcal{A}_{\bar{a}}$ gives $B \in \mathcal{A}$, $a \notin B$, $B \cap S = U$, and then $B \cap (\{a\} \cup S) = B \cap S = U$. If $a \in U$: let $U' = U \setminus \{a\} \subseteq S$; Q gives B with $B \cap S = U'$ and, since $B \in M$, $\{a\} \cup B \in \mathcal{A}$; then $(\{a\} \cup B) \cap (\{a\} \cup S) = \{a\} \cup (B \cap S) = \{a\} \cup U' = U$.

Putting it together. By (A), $\text{Tr}_{X'}(P) \subseteq \text{Tr}_X(\mathcal{A})$. By (B), $\{\{a\} \cup S : S \in \text{Tr}_{X'}(Q)\} \subseteq \text{Tr}_X(\mathcal{A})$. The first collection consists of sets avoiding a ; the second of sets containing a — so they are **disjoint**, and $S \mapsto \{a\} \cup S$ is injective on $\text{Tr}_{X'}(Q)$ (those sets avoid a). Therefore

$$|\text{Tr}_X(\mathcal{A})| \geq |\text{Tr}_{X'}(P)| + |\text{Tr}_{X'}(Q)| \stackrel{\text{IH}}{\geq} |P| + |Q| = |\mathcal{A}|. \quad \blacksquare$$

(II) Counting: $|\text{Tr}(\mathcal{A})| \leq \sum_{k=0}^d \binom{n}{k}$

Every shattered set S has $|S| \leq d$ by definition of d , and $S \subseteq X$. Thus $\text{Tr}(\mathcal{A})$ is contained in the family of subsets of X of size at most d , of which there are exactly $\sum_{k=0}^d \binom{n}{k}$. Monotonicity of cardinality finishes it.

Combining (I) and (II) gives the lemma. $\blacksquare$

2. The formalization (from scratch)

File: `Project/Project/SauerShelah.lean`, namespace `SauerShelah`. Ground type α with `[DecidableEq α]`; the ground set is an explicit $X : \text{Finset } \alpha$ (this is what makes induction on X possible — `mathlib`'s `shatterer` is ground-set-free and is *not* used).

Definitions (mirroring §0)

```
def Shatters (F : Finset (Finset α)) (T : Finset α) : Prop :=
  ∀ U ⊆ T, ∃ A ∈ F, A ∩ T = U                                -- "F shatters T"

def traces (X : Finset α) (F : Finset (Finset α)) : Finset (Finset α) :=
  X.powerset.filter (Shatters F)                                -- Tr_X(F), as a Finset
```

```
def vcDim (A : Finset (Finset α)) : ℕ := (traces univ A).sup Finset.card -- d
```

Shatters is given a `DecidablePred` instance (quantifying `U` over `T.powerset`) so that `traces` can be a `Finset` via `filter`. `mem_traces` unfolds membership to $S \subseteq X \wedge \text{Shatters } F \ S$.

Mapping paper $\rightarrow$ Lean

Paper step	Lean name / tactic
(I) Pajor’s lemma	<code>theorem pajor</code> , induction <code>X</code> using <code>Finset.induction</code>
Base case bookkeeping	<code>Finset.subset_singleton_iff</code> , explicit <code>traces $\emptyset \ \{\emptyset\} = \{\emptyset\}$</code>
$\mathcal{A}_a, \mathcal{A}_{\bar{a}}, M, P, Q$	<code>set Fa, Fn, M, P, Q</code> with ...
$ M = \mathcal{A}_a $ (delete- a injective)	<code>card_image_of_injOn + insert_erase</code>
$ \mathcal{A} = M + \mathcal{A}_{\bar{a}} $	<code>card_filter_add_card_filter_not</code>
$ \mathcal{A} = P + Q $	<code>card_union_add_card_inter</code>
Claim (A)	<code>claimA</code> , using <code>insert_inter_of_notMem</code>
Claim (B)	<code>claimB</code> , cases on $a \in U$, <code>insert_inter_of_mem</code> / <code>inter_insert_of_notMem</code> , <code>insert_erase</code>
disjoint + injective glue	<code>disjoint_left</code> , <code>card_image_of_injOn</code> , <code>card_union_of_disjoint</code>
final chain	a <code>calc</code> ending in <code>card_le_card (union_subset claimA claimB)</code>
(II) counting	<code>card_traces_le_sum</code> : <code>card_le_card</code> into <code>(Iic d).biUnion (powersetCard · univ)</code> , then <code>card_biUnion_le</code> , <code>card_powersetCard</code>
$\sum_{k \leq d}$ vs <code>range (d+1)</code>	<code>Nat.range_succ_eq_Iic</code>
final theorem	<code>card_le_sum_choose := (pajor univ A ...).trans (card_traces_le_sum A)</code>

The Lean proof is a *faithful transcription*: every `have` in `pajor` corresponds to a sentence in §1, and the file’s comments name the mathematical fact at each step. Only general-purpose `Finset/powersetCard` lemmas are used — **no** `Finset.shatterer`, `card_le_card_shatterer`, or `card_shatterer_le_sum_vcDim`.

3. Verification — convincing a human (and the kernel)

A proof that *compiles* can still be the wrong theorem. The following checks, all reproducible, establish that the artifact proves exactly the Sauer–Shelah lemma of §0. Build first:

```
cd Project
lake exe cache get      # fetch mathlib (first time only)
lake build               # compiles SauerShelah.lean; the kernel checks every proof
```

V1 — Read the statement. The conclusion of `card_le_sum_choose` is literally $\mathcal{A}.\text{card} \leq \sum k \in \text{Finset.range } (\text{vcDim } \mathcal{A} + 1), (\text{Fintype.card } \alpha).\text{choose } k$, i.e. $|\mathcal{A}| \leq \sum_{k=0}^d \binom{n}{k}$. There is **no extra hypothesis** on $\mathcal{A}$: it holds for *every* finite class, so the statement cannot be vacuously true.

V2 — The definitions are the intended ones. Shatters $F \ T$ is *verbatim* $\forall U \subseteq T, \exists A \in F, A \cap T = U$ (the §0 definition), and `vcDim` is the **sup** of cardinalities of shattered sets — the largest shattered set. Nothing is hidden behind a nonstandard definition. A grader compares these three **defs** against §0 directly.

V3 — No cheats. No `sorry/admit` appears:

```
grep -rn 'sorry\|admit' Project/Project/      # empty
```

and the kernel-level axiom trace of the main theorem is the standard `mathlib` base only:

```
#print axioms SauerShelah.card_le_sum_choose
-- 'SauerShelah.card_le_sum_choose' depends on axioms: [propext, Classical.choice, Quot.sound]
```

No `sorryAx`, no stray axiom, no `Lean.ofReduceBool` (so the theorem does **not** lean on `native_decide`). This line is printed on every build.

V4 — Non-vacuous, tight test instances (these examples are checked by the compiler): - Empty class $\emptyset$ over `Fin 5`: the bound applies (degenerate $0 \leq \dots$). - Full powerset over `Fin 4`: `vcDim = 4` is confirmed by computation (example : `vcDim ((univ : Finset (Fin 4)).powerset) = 4 := by native_decide`), catching any `vcDim` that silently computes the wrong number. Here the bound is $2^4 = 16 \leq \sum_{k=0}^4 \binom{4}{k} = 16$ — **equality**, so the lemma is tight and the inequality is not slack-by-construction.

V5 — Independence. A reviewer can confirm the reproof claim mechanically: the file never imports `mathlib`'s Sauer–Shelah / compression modules, so it *cannot* be silently leaning on them. (The strings `card_le_card_shatterer` etc. appear only in the header comment that states this very fact.)

```
grep -n '^import' Project/Project/SauerShelah.lean
# → Finset.Powerset, Fintype.Powerset, Interval.Finset.Nat, BigOperators only;
# no Combinatorics.SetFamily.Shatter and no .Compression.Down
```

Passing V1–V5 means: the right statement (V1–V2), genuinely proved (V3), neither vacuous nor loose (V4), and from scratch (V5).

Files

```
Project/
├─ REPORT.md           ← this document
├─ CLAUDE.md           ← project guidance (proof presentation + validation policy)
├─ lean-toolchain      ← Lean v4.30.0
├─ lakefile.toml       ← mathlib v4.30.0 dependency
├─ Project.lean        ← root import
└─ Project/SauerShelah.lean ← definitions, Pajor induction, counting, theorem, tests
```